გაკვეთილი 25 mod 3 (3.4)

გაკვეთილი 4 — ტაიმერის ფუნქციები JavaScript-ში

1. რა არის ტაიმერის ფუნქციები?

JavaScript-ში პროგრამის მოქმედებები ჩვეულებრივ სრულდება კოდის შესრულების პროცესში.

მაგალითად:

```
console.log("First");
console.log("Second");
console.log("Third");
```

შედეგი:

```
First
Second
Third
```

მაგრამ ზოგჯერ გვჭირდება, რომ გარკვეული მოქმედება დროის გასვლის შემდეგ შესრულდეს, ან ავტომატურად განმეორდეს.

მაგალითად:

- შეტყობინება გამოჩნდეს 3 წამის შემდეგ;
- საათი ყოველ წამში განახლდეს;
- წამმზომი ავტომატურად ითვლიდეს წამებს;
- სურათი ყოველ 5 წამში იცვლებოდეს;
- უკუთვლის ტაიმერი 10-დან 0-მდე ითვლიდეს.

ამისთვის JavaScript-ში არსებობს ტაიმერის ფუნქციები.

ამ გაკვეთილში ვისწავლით სამ ძირითად ფუნქციას:

ფუნქცია	დანიშნულება
`setTimeout()`	მოქმედებას ერთხელ ასრულებს მითითებული დროის გასვლის შემდეგ
`setInterval()`	მოქმედებას პერიოდულად იმეორებს

ფუნქცია	დანიშნულება
`clearInterval()`	აჩერებს `setInterval()`-ით დაწყებულ განმეორებას

2. `setTimeout()` — მოქმედების გადავადება

`setTimeout()` გამოიყენება მაშინ, როდესაც გვინდა, რომ კონკრეტული მოქმედება შესრულდეს გარკვეული დროის გასვლის შემდეგ.

მისი სინტაქსია:

```
setTimeout(function, milliseconds);
```

სადაც:

- `function` — ფუნქცია, რომელიც უნდა შესრულდეს;
- `milliseconds` — დრო მილიწამებში.

1000 მილიწამი = 1 წამი.

მაგალითად:

```
setTimeout(function() {
    console.log("Hello!");
}, 3000);
```

ეს კოდი Console-ში გამოიტანს `"Hello!"` შეტყობინებას დაახლოებით 3 წამის შემდეგ.

როგორ მუშაობს?

1. JavaScript შეხვდება `setTimeout()` ფუნქციას.
2. ბრაუზერი იწყებს დროის ათვლას.
3. დაახლოებით 3000 მილიწამის შემდეგ ფუნქცია მზად არის შესასრულებლად.
4. JavaScript მას შეასრულებს, როდესაც შესრულების გარემო ამის საშუალებას მისცემს.

მნიშვნელოვანია: `setTimeout()` დანარჩენი JavaScript კოდის შესრულებას არ აჩერებს.

მაგალითი 1 — შესრულების თანმიმდევრობა

```
console.log("Start");

setTimeout(function() {
    console.log("After 2 seconds");
```

```
}, 2000);

console.log("End");
```

შედეგი:

```
Start
End

(2 წამის შემდეგ)
After 2 seconds
```

რატომ?

იმიტომ, რომ `setTimeout()` არ ელოდება ორი წამის გასვლას ისე, რომ დანარჩენი კოდი გააჩეროს. ის გეგმავს ფუნქციის მოგვიანებით შესრულებას და პროგრამა აგრძელებს მუშაობას.

მაგალითი 2 — Arrow Function

რადგან მოსწავლეებმა ფუნქციები უკვე იციან, შეგვიძლია იგივე მაგალითი უფრო მოკლედ დავწეროთ.

```
setTimeout(() => {
    console.log("Hello after 3 seconds!");
}, 3000);
```

ეს ზუსტად იგივე პრინციპით მუშაობს.

მაგალითი 3 — HTML ელემენტის შეცვლა 3 წამის შემდეგ

მოცემულია:

```
<h1 id="title">Please wait...</h1>
```

JavaScript:

```
setTimeout(function() {
    let title = document.getElementById("title");

    title.textContent = "Welcome!";
}, 3000);
```

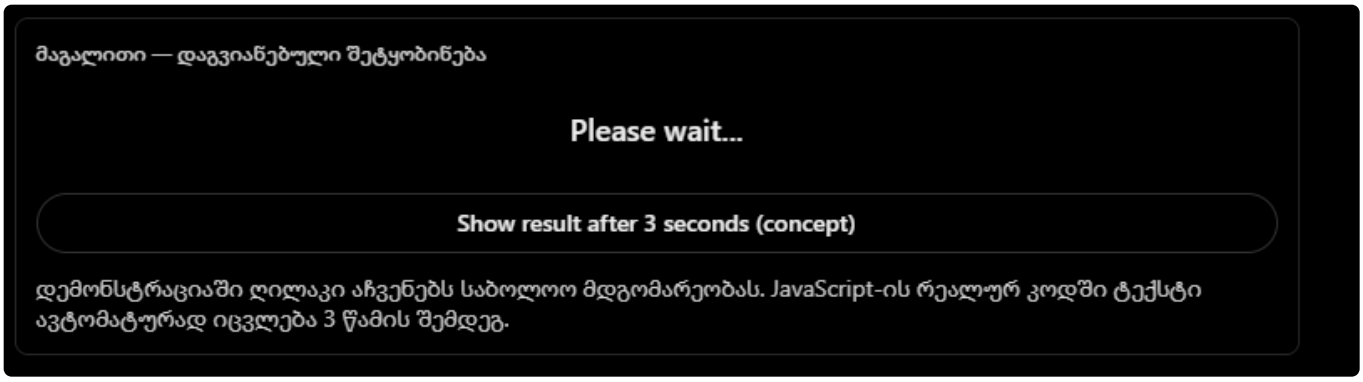
გვერდის გახსნისას გამოჩნდება:

```
Please wait...
```

სამი წამის შემდეგ კი:

```
Welcome!
```

მაგალითი — დაგვიანებული შეტყობინება



3. `setInterval()` — მოქმედების პერიოდულად გამეორება

`setInterval()` გამოიყენება მაშინ, როდესაც გვინდა, რომ მოქმედება განმეორდეს დროის გარკვეული ინტერვალით.

სინტაქსი:

```
setInterval(function, milliseconds);
```

მაგალითად:

```
setInterval(function() {
    console.log("Hello!");
}, 1000);
```

ამ შემთხვევაში Console-ში დაახლოებით ყოველ წამში გამოჩნდება:

```
Hello!
Hello!
Hello!
```

```
Hello!
...
```

ეს გაგრძელდება მანამ, სანამ ინტერვალს არ შევაჩერებთ ან შესაბამისი გვერდის/გარემოს მუშაობა არ დასრულდება.

მაგალითი 1 — რიცხვების ზრდა

```
let number = 0;

setInterval(function() {
    number++;

    console.log(number);
}, 1000);
```

შედეგი:

```
1
2
3
4
5
...
```

აქ:

1. თავდაპირველად `number = 0`;
2. ყოველ წამში შესრულდება ფუნქცია;
3. `number++` გაზრდის მნიშვნელობას ერთით;
4. ახალი მნიშვნელობა გამოჩნდება Console-ში.

4. `clearInterval()` — ინტერვალის შეჩერება

როდესაც `setInterval()`-ს ვიყენებთ, ზოგჯერ გვჭირდება მისი შეჩერება.

ამისთვის არსებობს:

```
clearInterval()
```

მაგრამ ჯერ უნდა ვიცოდეთ, რომ `setInterval()` აბრუნებს ინტერვალის იდენტიფიკატორს.

მაგალითად:

```
let timer = setInterval(function() {
    console.log("Running...");
}, 1000);
```

`timer` ცვლადში ინახება იდენტიფიკატორი, რომლითაც შეგვიძლია ეს ინტერვალი შევაჩეროთ.

```
clearInterval(timer);
```

მაგალითი — ათვლა 5-მდე

შევქმნათ პროგრამა, რომელიც ყოველ წამში გაზრდის რიცხვს, ხოლო 5-ზე შეჩერდება.

```
let number = 0;

let timer = setInterval(function() {
    number++;

    console.log(number);

    if (number === 5) {
        clearInterval(timer);
    }
}, 1000);
```

შედეგი:

```
1
2
3
4
5
```

ამის შემდეგ პროგრამა აღარ გააგრძელებს რიცხვების გამოტანას.

მნიშვნელოვანი: `clearInterval()` მხოლოდ მითითებულ ინტერვალს აჩერებს და არა მთელ JavaScript პროგრამას.

5. `setTimeout()` და `setInterval()` — განსხვავება

ეს ორი ფუნქცია ერთმანეთის მსგავსია, თუმცა სხვადასხვა დანიშნულება აქვს.

`setTimeout()`	`setInterval()`
ფუნქციას ერთხელ ასრულებს	ფუნქციას პერიოდულად ასრულებს
გამოიყენება მოქმედების გადავადებისთვის	გამოიყენება განმეორებითი მოქმედებებისთვის
მაგალითად, შეტყობინება 3 წამის შემდეგ	მაგალითად, საათის განახლება ყოველ წამში

მაგალითი

```
setTimeout(function() {
    console.log("Hello");
}, 2000);
```

`Hello` ერთხელ გამოჩნდება.

ხოლო:

```
setInterval(function() {
    console.log("Hello");
}, 2000);
```

`Hello` დაახლოებით ყოველ ორ წამში გამოჩნდება.

6. პრაქტიკული მაგალითი — წამმზომი (Stopwatch)

ახლა შევქმნათ რეალური ვებგვერდი, სადაც მომხმარებელს შეეძლება წამმზომის დაწყება და შეჩერება.

ამისთვის გამოვიყენებთ:

- `setInterval()`;
- `clearInterval()`;
- `getElementById()`;
- `textContent`;
- ღილაკებსა და ფუნქციებს.

სრული კოდი

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Stopwatch</title>
</head>
<body>

    <h1>Stopwatch</h1>

    <h2 id="time">0</h2>

    <button onclick="startTimer()">Start</button>
    <button onclick="stopTimer()">Stop</button>
    <button onclick="resetTimer()">Reset</button>

    <script>
        let seconds = 0;
        let timer = null;

        function startTimer() {
            if (timer !== null) {
                return;
            }

            timer = setInterval(function() {
                seconds++;

                document.getElementById("time").textContent = seconds;
            }, 1000);
        }

        function stopTimer() {
            clearInterval(timer);
            timer = null;
        }

        function resetTimer() {
            stopTimer();

            seconds = 0;

            document.getElementById("time").textContent = seconds;
        }
    </script>

</body>
</html>
```

როგორ მუშაობს?

`seconds` ცვლადი

```
let seconds = 0;
```

ინახავს გასული წამების რაოდენობას.

`timer` ცვლადი

```
let timer = null;
```

ინახავს ინტერვალის იდენტიფიკატორს. `null` ნიშნავს, რომ ამ მომენტში ინტერვალი გაშვებული არ გვაქვს.

Start ღილაკი

```
function startTimer() {
    if (timer !== null) {
        return;
    }

    timer = setInterval(function() {
        seconds++;

        document.getElementById("time").textContent = seconds;
    }, 1000);
}
```

ყოველ წამში ზრდის `seconds` ცვლადს და ცვლის HTML-ში ნაჩვენებ რიცხვს.

`if` ამოწმებს, ხომ არ მუშაობს წამმზომი უკვე. ასე თავიდან ვიცილებთ რამდენიმე ინტერვალის ერთდროულად გაშვებას.

Stop ღილაკი

```
function stopTimer() {
    clearInterval(timer);
    timer = null;
}
```

აჩერებს წამმზომს, მაგრამ გასულ წამებს არ შლის.

Reset ღილაკი

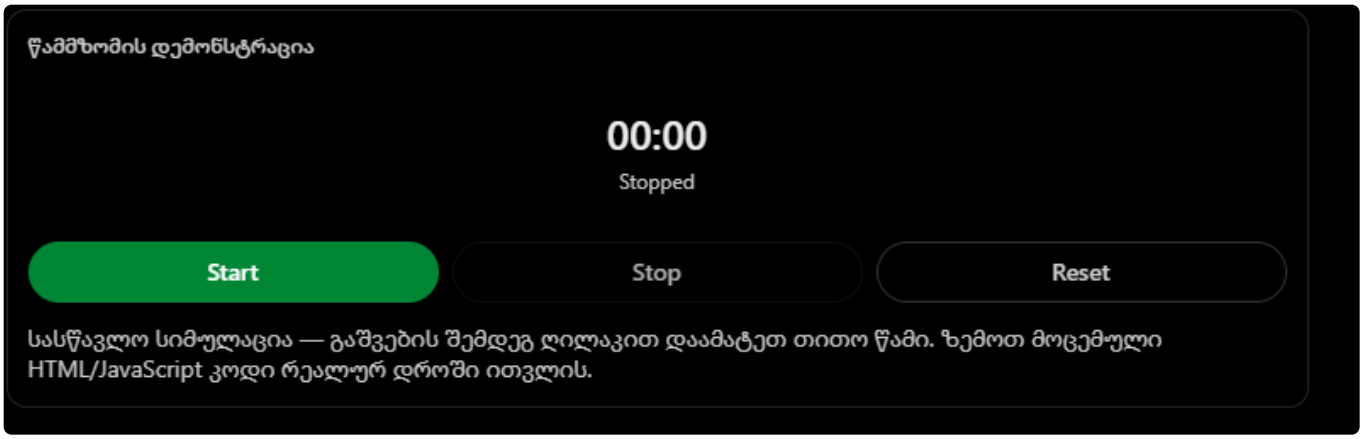
```
function resetTimer() {
    stopTimer();

    seconds = 0;

    document.getElementById("time").textContent = seconds;
}
```

აჩერებს წამმზომს და აბრუნებს 0-ზე.

წამმზომის დემონსტრაცია



სასწავლო სიმულაცია — გაშვების შემდეგ ღილაკით დაამატეთ თითო წამი. ზემოთ მოცემული HTML/JavaScript კოდი რეალურ დროში ითვლის.

7. პრაქტიკული მაგალითი — უკუთვლის ტაიმერი (Countdown)

ახლა გავაკეთოთ პროგრამა, რომელიც დაიწყებს 10-დან და ყოველ წამში ერთით შეამცირებს რიცხვს.

როდესაც მიაღწევს 0-ს, ტაიმერი უნდა გაჩერდეს.

```
let seconds = 10;

let timer = setInterval(function() {
    seconds--;

    console.log(seconds);

    if (seconds === 0) {
        clearInterval(timer);
```

```
        console.log("Time is up!");
    }
}, 1000);
```

შედეგი:

```
9
8
7
6
5
4
3
2
1
0
Time is up!
```

HTML-თან ერთად

```
<!DOCTYPE html>
<html>
<head>
    <title>Countdown</title>
</head>
<body>

    <h1>Countdown Timer</h1>

    <h2 id="count">10</h2>

    <button onclick="startCountdown()">Start</button>

    <script>
        let seconds = 10;
        let timer = null;

        function startCountdown() {
            if (timer !== null || seconds === 0) {
                return;
            }

            timer = setInterval(function() {
```

```
                seconds--;

                document.getElementById("count").textContent = seconds;

                if (seconds === 0) {
                    clearInterval(timer);
                    timer = null;

                    alert("Time is up!");
                }
            }, 1000);
        }
    </script>

</body>
</html>
```

ამ მაგალითში მოსწავლეები ერთად იყენებენ ტაიმერის ფუნქციებს, DOM-ს და `if` პირობას.

8. პრაქტიკული მაგალითი — ციფრული საათი

ტაიმერის ფუნქციები ხშირად გამოიყენება რეალურ დროში განახლებადი საათის შესაქმნელად.

ამისთვის გავაერთიანოთ პირველი და მეოთხე გაკვეთილებში ნასწავლი მასალა:

- `Date`;
- `getHours()`;
- `getMinutes()`;
- `getSeconds()`;
- `setInterval()`.

სრული კოდი

```
<!DOCTYPE html>
<html>
<head>
    <title>Digital Clock</title>
</head>
<body>

    <h1>Digital Clock</h1>

    <h2 id="clock"></h2>
```

```
    <script>
        function updateClock() {
            let now = new Date();

            let hours = now.getHours();
            let minutes = now.getMinutes();
            let seconds = now.getSeconds();

            let time = hours + ":" + minutes + ":" + seconds;

            document.getElementById("clock").textContent = time;
        }

        updateClock();

        setInterval(updateClock, 1000);
    </script>

</body>
</html>
```

ახსნა

პირველ რიგში ვქმნით `updateClock()` ფუნქციას, რომელიც იღებს მიმდინარე დროს.

```
let now = new Date();
```

შემდეგ ვიღებთ საათს, წუთსა და წამს:

```
let hours = now.getHours();
let minutes = now.getMinutes();
let seconds = now.getSeconds();
```

ამის შემდეგ ვაერთიანებთ ერთ სტრიქონში:

```
let time = hours + ":" + minutes + ":" + seconds;
```

და ვაჩვენებთ HTML ელემენტში:

```
document.getElementById("clock").textContent = time;
```

ბოლოს:

```
updateClock();

setInterval(updateClock, 1000);
```

პირველი გამოძახება საათს მაშინვე აჩვენებს, ხოლო `setInterval()` მას ყოველ წამში განაახლებს.

დამატებითი გაუმჯობესება

ამჟამად საათმა შეიძლება გამოიტანოს:

```
9:5:3
```

მაგრამ უფრო მოსახერხებელი იქნება:

```
09:05:03
```

ამისთვის შეგვიძლია გამოვიყენოთ `padStart()`:

```
let hours = String(now.getHours()).padStart(2, "0");
let minutes = String(now.getMinutes()).padStart(2, "0");
let seconds = String(now.getSeconds()).padStart(2, "0");
```

`padStart(2, "0")` სტრიქონის დასაწყისში ამატებს ნულს, თუ მისი სიგრძე ორ სიმბოლოზე ნაკლებია.

ეს უკვე პირველი გაკვეთილის სტრიქონულ ფუნქციებთან კავშირსაც აჩვენებს.

9. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — დაგვიანებული შეტყობინება

მარტივი

დაწერეთ პროგრამა, რომელიც Console-ში მაშინვე გამოიტანს:

```
Program started
```

ხოლო 5 წამის შემდეგ:

```
Welcome to JavaScript!
```

გამოიყენეთ `setTimeout()`.

სავარჯიშო 2 — რიცხვების ავტომატური ზრდა

მარტივი

შექმენით პროგრამა, რომელიც ყოველ 2 წამში გაზრდის რიცხვს ერთით.

ათვლა უნდა დაიწყოს 0-დან და შეჩერდეს 10-ზე.

მოსალოდნელი შედეგი:

```
1
2
3
4
5
6
7
8
9
10
```

გამოიყენეთ:

- `setInterval()`;
- `clearInterval()`;
- `if`.

სავარჯიშო 3 — ტექსტის ავტომატური შეცვლა

საშუალო

მოცემულია:

```
<h1 id="message">Loading...</h1>
```

შექმენით JavaScript პროგრამა, რომელიც 3 წამის შემდეგ ტექსტს შეცვლის:

```
Loading complete!
```

გამოიყენეთ `setTimeout()` და `textContent`.

სავარჯიშო 4 — ფონის ფერის ავტომატური შეცვლა

საშუალო

შექმენით ვებგვერდი, რომლის ფონის ფერი ავტომატურად შეიცვლება ყოველ 2 წამში.

ფერები:

```
let colors = ["red", "blue", "green", "yellow"];
```

მოთხოვნები:

1. გამოიყენეთ `setInterval()`;
2. ყოველ 2 წამში გადადით მასივის შემდეგ ფერზე;
3. ბოლო ფერის შემდეგ დაიწყეთ ისევ პირველი ფერიდან;
4. დაამატეთ Stop ღილაკი, რომელიც ფერების შეცვლას შეაჩერებს.

სავარჯიშო 5 — უკუთვლის ტაიმერი

საშუალო

შექმენით HTML გვერდი, რომელზეც გამოჩნდება რიცხვი `20`.

გვერდზე უნდა იყოს ორი ღილაკი:

- Start;
- Stop.

Start ღილაკზე დაჭერისას რიცხვი ყოველ წამში უნდა შემცირდეს ერთით.

როდესაც მიაღწევს 0-ს, გამოჩნდეს შეტყობინება:

```
Time is up!
```

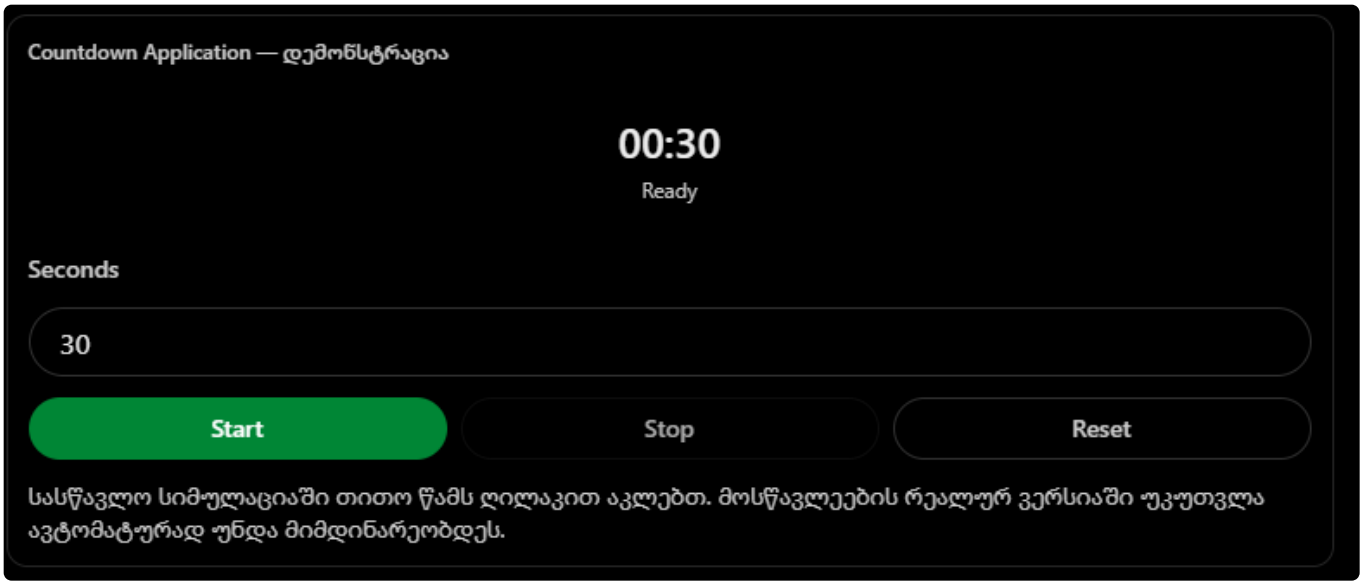
Stop ღილაკზე დაჭერისას ათვლა უნდა შეჩერდეს.

10. შემაჯამებელი პრაქტიკული დავალება — Countdown Application

რთული დავალება

შექმენით პატარა ვებაპლიკაცია, სადაც მომხმარებელი თავად მიუთითებს, რამდენი წამიდან უნდა დაიწყოს უკუთვლა.

Countdown Application — დემონსტრაცია



სასწავლო სიმულაციაში თითო წამს ღილაკით აკლებთ. მოსწავლეების რეალურ ვერსიაში უკუთვლა ავტომატურად უნდა მიმდინარეობდეს.

დავალების მოთხოვნები

1. მომხმარებელმა `input` ველში უნდა შეიყვანოს წამების რაოდენობა.
2. Start ღილაკზე დაჭერისას უნდა დაიწყოს უკუთვლა.
3. ყოველ წამში რიცხვი უნდა შემცირდეს ერთით.
4. Stop ღილაკმა უნდა შეაჩეროს უკუთვლა.
5. Reset ღილაკმა უნდა დააბრუნოს საწყისი მნიშვნელობა.
6. როდესაც დრო მიაღწევს 0-ს, გამოჩნდეს `"Time is up!"`.
7. პროგრამამ არ უნდა დაუშვას ერთდროულად რამდენიმე ინტერვალის გაშვება.
8. უარყოფითი და არასწორი შეყვანილი მნიშვნელობები უნდა დამუშავდეს.

საწყისი HTML კოდი

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Countdown App</title>
</head>
<body>

    <h1>Countdown Application</h1>

    <input type="number" id="secondsInput" placeholder="Enter seconds">

    <h2 id="display">0</h2>
```

```
    <button onclick="startTimer()">Start</button>
    <button onclick="stopTimer()">Stop</button>
    <button onclick="resetTimer()">Reset</button>

    <script>
        let timer = null;
        let seconds = 0;

        function startTimer() {
            // დაწერეთ კოდი აქ
        }

        function stopTimer() {
            // დაწერეთ კოდი აქ
        }

        function resetTimer() {
            // დაწერეთ კოდი აქ
        }
    </script>

</body>
</html>
```

მოსწავლეებმა დამოუკიდებლად უნდა გამოიყენონ:

```
setInterval()
clearInterval()
getElementById()
textContent
if / else
```